Game

Actions

Actions and Events

Action

What an **Entity** does for its turn

Event

Something that happens

Example: **Entity** produces an Attack **Action** → triggers a Hit **Event** (defender takes damage)

class Action

```
// base class for all actions
class Action {
public:
    virtual ~Action() {}

    // override perform in a derived class
    virtual Result perform(Engine& engine, std::shared_ptr<Entity> entity) = 0;
};
```

You only need to override `perform()`

class Action

```cpp
// base class for all actions
class Action {
public:
    virtual ~Action() {}

    // override perform in a derived class
    virtual Result perform(Engine& engine, std::shared_ptr<Entity> entity) = 0;
};
```

- What pieces of data can you access inside `perform()`?
- `entity`, full game `engine`, any data in derived classes

Results

Rest action in content/actions/rest.h

```
class Rest : public Action {
public:
    Result perform(Engine& engine, std::shared_ptr<Entity> entity) override;
};
```

Performing an **Action** produces a **Result**

Results

In `engine/action.h`

```
// the result of performing an Action
class Result {
public:
    bool succeeded{false};
    std::unique_ptr<Action> next_action{nullptr}; // allows chaining of actions
};
```

`succeeded`

Whether **Action** could be performed

`next_action`

Action to be performed after previous one

Results

You do not produce Results directly, use helper functions!

`success()`

Action worked, **Entity** finished with turn

`failure()`

Couldn't perform **Action**, give **Entity** another turn

`alternative(AnotherAction{})`

Couldn't perform **Action**, do this one instead!

Example Action-Result

```
// content/actions/rest.h
class Rest : public Action {
public:
    Result perform(Engine& engine, std::shared_ptr<Entity> entity) override;
};

// content/actions/rest.cpp
Result Rest::perform(Engine&, std::shared_ptr<Entity>) {
    return success();
}
```

Example Action-Result

- When to use `failure()` or `alternative()`?
- Consider a general Move **Action** which attempts to place an **Entity** at a new position:

```
If new position is wall
    return failure()

If new position is door
    return alternative(OpenDoor{})
```

Another example

To make Monsters wander around the dungeon, a **Wander** action could do

```
for each neighboring tile:
    if tile is walkable and has no entity:
        return alternative(Move{})

// no empty tile to move to -> rest this turn
return alternative(Rest{})
```

- BUG: Returning failure() would give the monster another turn → infinite loop

Moving

You must understand these classes

Vec

(x, y) tile location, (0, 0) is bottom-left

Entity

public member functions

Tiles

```
Tile& tile =
engine.dungeon.get_tile(position)
```

Tile

```
is_wall(),bool walkable,Entity*
entity
```

Vec

A 2D vector class used for position and direction

```
// engine/util/vec.h
class Vec {
public:
    int x{0}, y{0};
};
```

Arithmetic operators:

```
Vec operator+(const Vec& a, const Vec& b);
Vec operator-(const Vec& a, const Vec& b);
```

Vec

Example usage

```
Vec position{10, 10}; // (x, y)
Vec direction{1, 0};  // right
Vec new_position = position + direction; // (11, 10)
// OR
Vec old_pos{2, 3}, new_pos{3, 3};
Vec direction = new_pos - old_position; // (1, 0)
```

Vec

Directions

x, y	direction
1, 0	right
-1, 0	left
0, 1	up
0, -1	down

Entity

For now only consider these memfuncs:

```
// engine/entity.h
class Entity {
public:
    Vec get_position() const;
    void move_to(Vec position);
    void change_direction(Vec direction);
};
```

Tiles

- A 2D grid of class **Tile** inside class **Dungeon** (`engine/dungeon/dungeon.h`)

```
Vec position{1, 2};
Tile& tile = engine.dungeon.get_tile(position);
// do things with tile
```

Tile

Square tile within dungeon

```
class Tile {
public:
    Tile();
    bool is_wall() const;
    bool has_door() const;
    bool has_item() const;
    bool has_entity() const;
    bool is_visible() const;

    enum class Type { None, Floor, Wall, Door };

    Type type;
    Sprite sprite;
    bool visible;
    bool walkable;

    std::shared_ptr<Door> door;
    std::shared_ptr<Item> item;
    Entity* entity;
```

Tile

```
// focus on these only
class Tile {
public:
    bool is_wall() const;
    bool has_door() const;
    bool has_entity() const;
};
```

```
Vec pos{1, 1};
Tile& tile = engine.dungeon.get_tile(pos);
if (tile.is_wall() || tile.has_entity()) {
    // cannot move there
}

if (tile.has_door()) {
    // do something with door
}
```

Taking turns

The engine calls `take_turn()` for each `Entity`

```
std::unique_ptr<Action> Entity::take_turn() {
    if (behavior) {
        return behavior(engine, *this);
    }
    return nullptr;
}
```

`behavior` is a function that returns an `Action`

```
class Entity {
public:
    // taking turns
    std::unique_ptr<Action> take_turn();
    std::function<std::unique_ptr<Action>(Engine& engine, Entity& entity)> behavior;
};
```

Keybindings

A human-controlled entity could have this behavior:

```
// content/entities/heroes.cpp
std::unique_ptr<Action> behavior(Engine& engine, Entity&) {
    std::string key = engine.input.get_last_keypress();
    if (key == "Right") {
        return std::make_unique<Move>(Vec{1, 0});
    }
    else if (key == "F") {
        return std::make_unique<ThrowFire>();
    }
    return nullptr;
}
```

AI

A computer-controlled entity could have this behavior:

```
// content/entities/monsters.cpp
std::unique_ptr<Action> behavior(Engine& engine, Entity& entity) {
    if (entity.is_visible() && engine.hero) {
        std::vector<Vec> path = engine.dungeon.calculate_path(entity.get_position
                                                              engine.hero->get_pos
        if (path.size() > 1) {
            Vec direction = path.at(1) - path.at(0);
            return std::make_unique<Move>(direction);
        }
    }

    if (probability(66)) {
        return std::make_unique<Wander>();
    }
    else {
        return std::make_unique<Rest>();
    }
}
```

Adding behaviors

Simply set the behavior of an Entity when you are customizing it

```
void make_wizard(std::shared_ptr<Entity>& hero) {
    hero->set_sprite("wizard");
    hero->set_max_health(10);
    hero->behavior = behavior;
}
```

Adding behaviors

```
int main() {
    try {
        Settings settings{"settings.txt"};
        Engine engine{settings};

        std::shared_ptr<Entity> hero = engine.create_hero();
        Heroes::make_wizard(hero);

        engine.run();
    }
    catch (std::exception& e) {
        std::cout << e.what() << '\n';
    }
}
```

Checkpoint 1

- Create **Move** action
  - Only move Hero to an empty tile (`success()`)
  - Walls, doors, and other entities block (`failure()`)
  - Don't forget to set direction of entity
- Bind **Move** to up/down/left/right or WASD
- Bind **Rest** action to a key of your choice